

Sommaire

AP3 — Clash of League (front-end + back-end)	1
1. Présentation	1
2. Objectifs	2
3. Stack technique	2
4. Architecture	3
5. Présentation des pages principales	3
6. Gestion de projet — Kanban GitHub Projects	6
7. Difficultés rencontrées	7
8. Compétences BTS SIO mises en œuvre	7
9. Bilan	8
10. Liens utiles	8

AP3 — Clash of League (front-end + back-end)

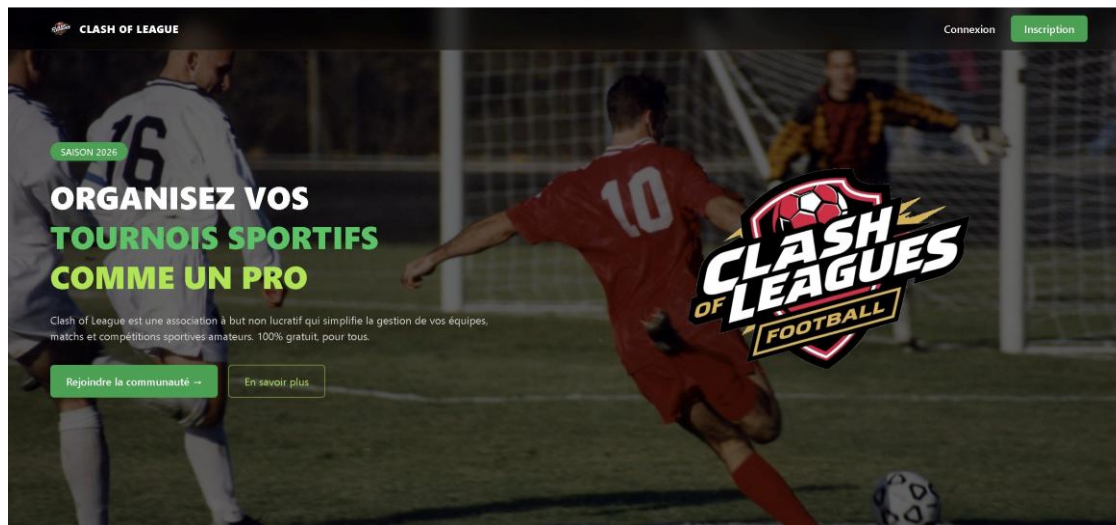
Auteur : William TOUSSON · BTS SIO SLAM 2^e année · IPSSI SQY **Période :**
03/09/2025 – 19/05/2026 **Type :** Activité Professionnelle 3 (AP3) — projet
d'équipe **Équipe :** William TOUSSON · Victor COMBLEMORELBLUNTZ · Loick
DEVISMES **Rôle de William :** principalement le front-end **Dépôts :** - Front-end :
<https://github.com/victorcb2003/ClashFrontEnd> - Back-end :
<https://github.com/victorcb2003/ClashBackEnd>

1. Présentation

Application web complète pour l'association fictive « **Clash of League** », qui simplifie la gestion d'équipes, de matchs et de tournois sportifs amateurs. Le site permet à un utilisateur connecté de **créer ou rejoindre une équipe, consulter le calendrier des matchs, organiser des tournois et gérer son profil.**

Le projet est découpé en **deux applications distinctes** qui communiquent ensemble :

- une **application front-end** en **React** (interface utilisateur, navigation, formulaires) ;
- une **API back-end** en **Node.js / Express** avec une **base de données MySQL** (logique métier, authentification, persistance des données).



Page d'accueil publique

2. Objectifs

- Concevoir et développer une **application complète client/serveur**.
- Maîtriser **React** (composants, hooks, routing) et l'écosystème **Vite**.
- Construire une **API REST** en Node.js / Express avec accès MySQL.
- Mettre en place une **authentification sécurisée** (JWT, bcrypt).
- Travailler en **équipe sur deux dépôts** distincts avec coordination front / back.
- Gérer le projet avec un outil professionnel : **GitHub Projects** (Kanban).

3. Stack technique

3.1 Front-end (ClashFrontEnd)

Composant	Détail
React	19.2
Vite	7.2 (build tool et serveur de dev)
React Router DOM	7.13 (routing client-side avec BrowserRouter)
Axios	1.13 (appels HTTP vers l'API + intercepteur JWT)
Tailwind CSS	3.4 (avec PostCSS et Autoprefixer)
React Calendar	6.0 (composant calendrier)
React Icons	5.5 (icônes de la sidebar)
dotenv	gestion des variables d'environnement

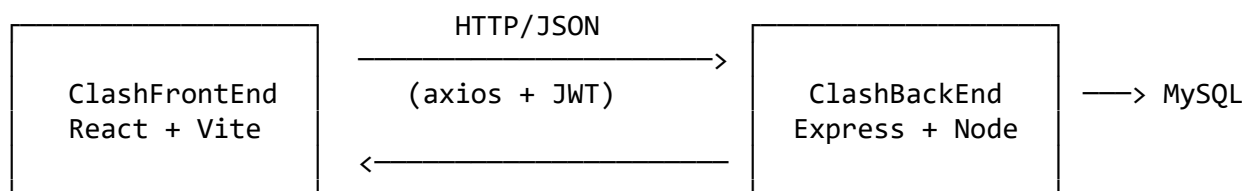
3.2 Back-end (ClashBackEnd)

Composant	Détail
Node.js + Express	5.1 (serveur HTTP + routing)

Composant	Détail
MySQL2	3.15 (driver MySQL pour Node)
bcrypt	6.0 (hash des mots de passe)
jsonwebtoken	9.0 (authentification par JWT)
cors	2.8 (gestion des appels cross-origin)
multer	2.0 (upload de fichiers — avatars, logos d'équipe)
nodemailer	7.0 (envoi d'emails)
handlebars	4.7 (templating des emails)

4. Architecture

Architecture **client-serveur classique** : le front React (SPA servie par Vite) appelle l'API REST via Axios, qui interroge la base MySQL et renvoie du JSON.



Côté front, un **intercepteur Axios** (`src/services/interceptor.js`) attache automatiquement le **token JWT** à chaque requête sortante et redirige vers la page de connexion en cas de token expiré.

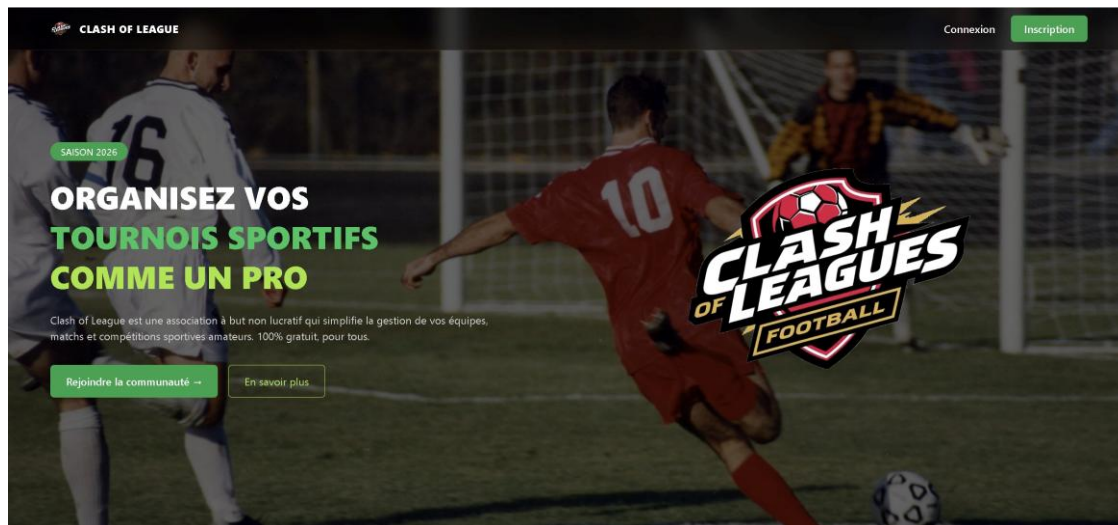
Côté back, un **middleware custom** (`app/middleware/decrypteToken`) déchiffre le JWT à chaque requête entrante pour identifier l'utilisateur, et un autre (`app/middleware/log`) journalise les appels. Les routes sont centralisées dans `app/routes/index`.

5. Présentation des pages principales

Le front-end compte **13 routes**, regroupées en deux catégories : pages publiques (Accueil, Connexion, Inscription, Réinitialisation de mot de passe) et pages privées accessibles après authentification.

5.1 Accueil public

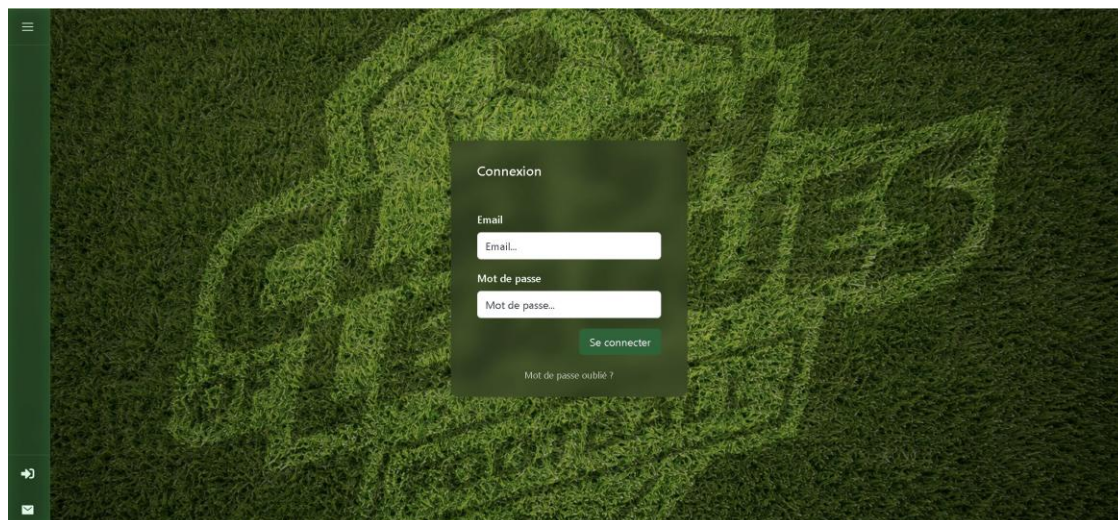
Page d'arrivée présentant l'association et invitant à rejoindre la communauté ou à se connecter.



Accueil public

5.2 Connexion

Formulaire de connexion par **email + mot de passe**, avec lien « Mot de passe oublié ? » qui déclenche l'envoi d'un email de réinitialisation (via Nodemailer côté back-end).



Page de connexion

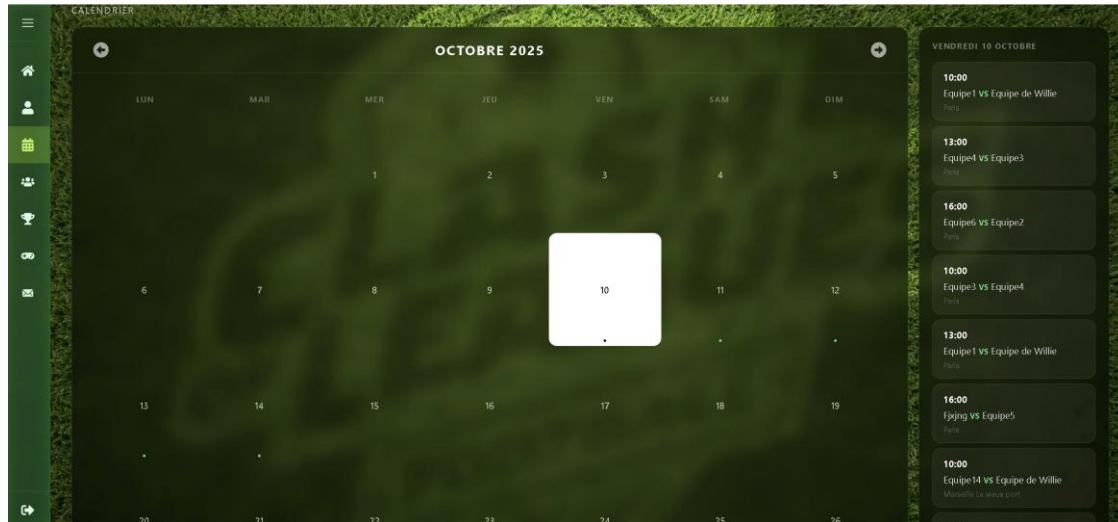
5.3 Équipes

Page listant toutes les équipes existantes, chacune avec son nom et son nombre de joueurs. Permet à l'utilisateur de consulter une équipe en détail (route /equipe/:id).

Capture à intégrer ultérieurement.

5.4 Tournois

Page listant les tournois passés, en cours et à venir, avec leur stade et leur date de début. Un bouton « **Créer un événement** » permet aux utilisateurs autorisés d'organiser un nouveau tournoi.



Page Tournois

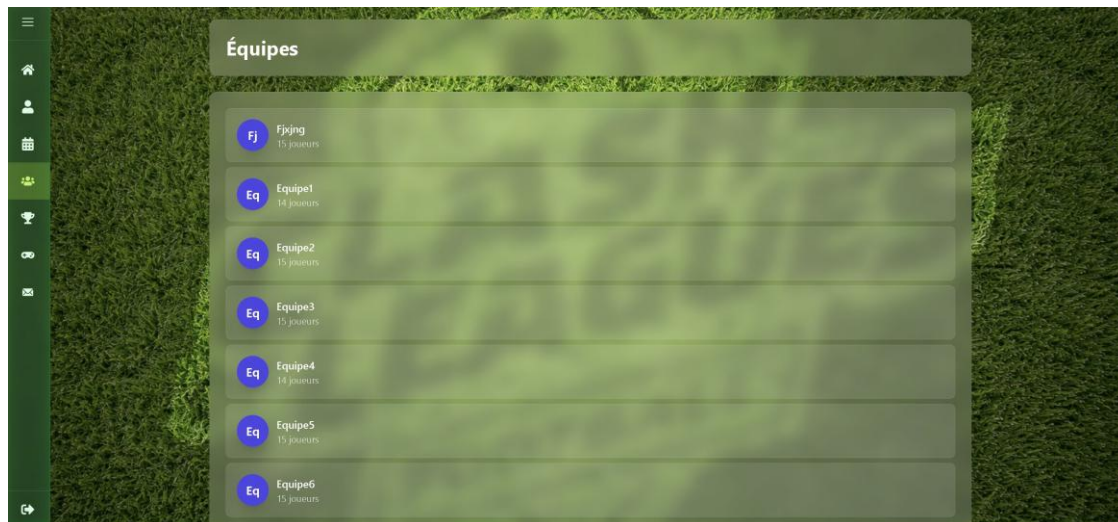
5.5 Calendrier

Page affichant un **calendrier mensuel** (composant react-calendar) avec les matchs programmés. Un panneau latéral détaille les matchs de la journée sélectionnée (équipes en confrontation, heure, lieu).

Capture à intégrer ultérieurement.

5.6 Matches

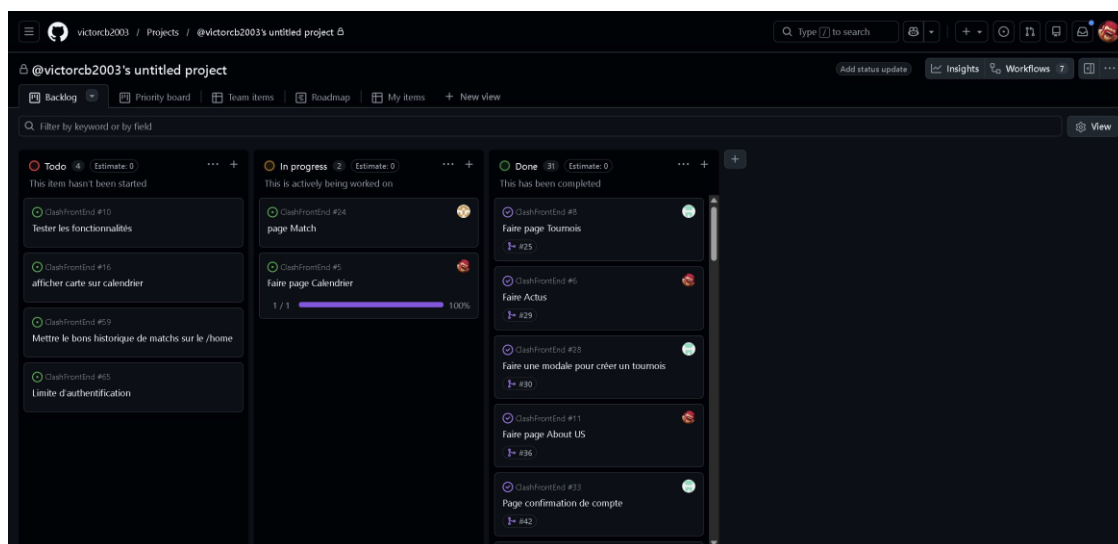
Liste chronologique des matchs (passés et à venir) avec les équipes, le lieu et l'horodatage précis. Bouton « + » pour ajouter un nouveau match.



Page Matches

6. Gestion de projet — Kanban GitHub Projects

L'équipe a piloté le développement avec **GitHub Projects** sur le dépôt front-end, en utilisant un tableau Kanban à trois colonnes : *Todo*, *In Progress*, *Done*.



Kanban GitHub Projects

Au moment de la capture : **4 tâches à faire**, **2 en cours** et **31 terminées**. Les tickets couvrent toute la production : « *Faire page Tournois* », « *Faire une modale pour créer un tournois* », « *Faire page About US* », « *Page confirmation de compte* », « *Faire page Calendrier* », « *afficher carte sur calendrier* », « *Limite d'authentification* », etc.

Cet outil permet à la fois la **planification** (qui fait quoi, quand) et la **gestion des incidents et demandes** (un bug ou une demande d'évolution devient un ticket à traiter dans le board).

7. Difficultés rencontrées

Les difficultés classiques d'un projet d'équipe sur plusieurs mois ont été rencontrées :

- **Coordination Git** : gestion des branches, résolution des conflits de merge entre plusieurs développeurs travaillant sur le même dépôt.
 - **Communication front / back** : se mettre d'accord sur le format précis des routes API et des données échangées (noms de champs, codes HTTP, gestion des erreurs).
 - **Authentification par JWT** : comprendre le cycle complet token (génération côté back, stockage côté front, attachement automatique aux requêtes, expiration et refresh).
 - **Apprentissage de React** : premier projet de cette envergure, prise en main des hooks (useState, useEffect), du routing client et des composants réutilisables.
 - **Responsive design** : adapter l'interface aux différents formats d'écran avec Tailwind CSS.
 - **Gestion des dates et du calendrier** : intégration du composant react-calendar et synchronisation avec les données du back-end.
-

8. Compétences BTS SIO mises en œuvre

Ce projet justifie quatre compétences du **Bloc 1** :

- **C2 — Répondre aux incidents et aux demandes d'assistance et d'évolution** → Sous-compétences : « *Collecter, suivre et orienter des demandes* » + « *Traiter des demandes concernant les applications* ». Le tableau Kanban GitHub Projects matérialise cette compétence : chaque bug détecté ou demande d'évolution devient un ticket prioritaire à traiter (cf. §6).
- **C3 — Développer la présence en ligne de l'organisation** → Sous-compétence : « *Participer à l'évolution d'un site Web exploitant les données de l'organisation* ». L'interface React est entièrement alimentée par les données servies par l'API back-end.
- **C4 — Travailler en mode projet** → Sous-compétences : « *Analyser les objectifs et les modalités d'organisation d'un projet* » + « *Planifier les activités* ». Projet long (~9 mois) avec architecture client/serveur impliquant coordination entre plusieurs développeurs et deux dépôts.
- **C5 — Mettre à disposition des utilisateurs un service informatique** → Sous-compétences : « *Déployer un service* » + « *Accompagner les utilisateurs dans la mise en* ».

place d'un service ». Mise à disposition d'un service informatique complet (front + API).

9. Bilan

Ce que ce projet m'a apporté :

- Premier projet **complet client-serveur** avec deux dépôts distincts à maintenir.
 - Apprentissage approfondi de **React** (composants, hooks, routing, gestion d'état).
 - Pratique de **Axios** avec un intercepteur global pour gérer l'authentification.
 - Compréhension du cycle complet d'une **API REST** sécurisée par JWT.
 - Travail en équipe sur **GitHub** avec un vrai workflow professionnel (branches, pull requests, code review, tickets).
 - Découverte de **Tailwind CSS** sur un projet conséquent.
-

10. Liens utiles

- Dépôt Front-end : <https://github.com/victorcb2003/ClashFrontEnd>
- Dépôt Back-end : <https://github.com/victorcb2003/ClashBackEnd>
- Documentation React : <https://react.dev/>
- Documentation Vite : <https://vitejs.dev/>
- Documentation Express : <https://expressjs.com/fr/>
- Documentation Tailwind CSS : <https://tailwindcss.com/docs>